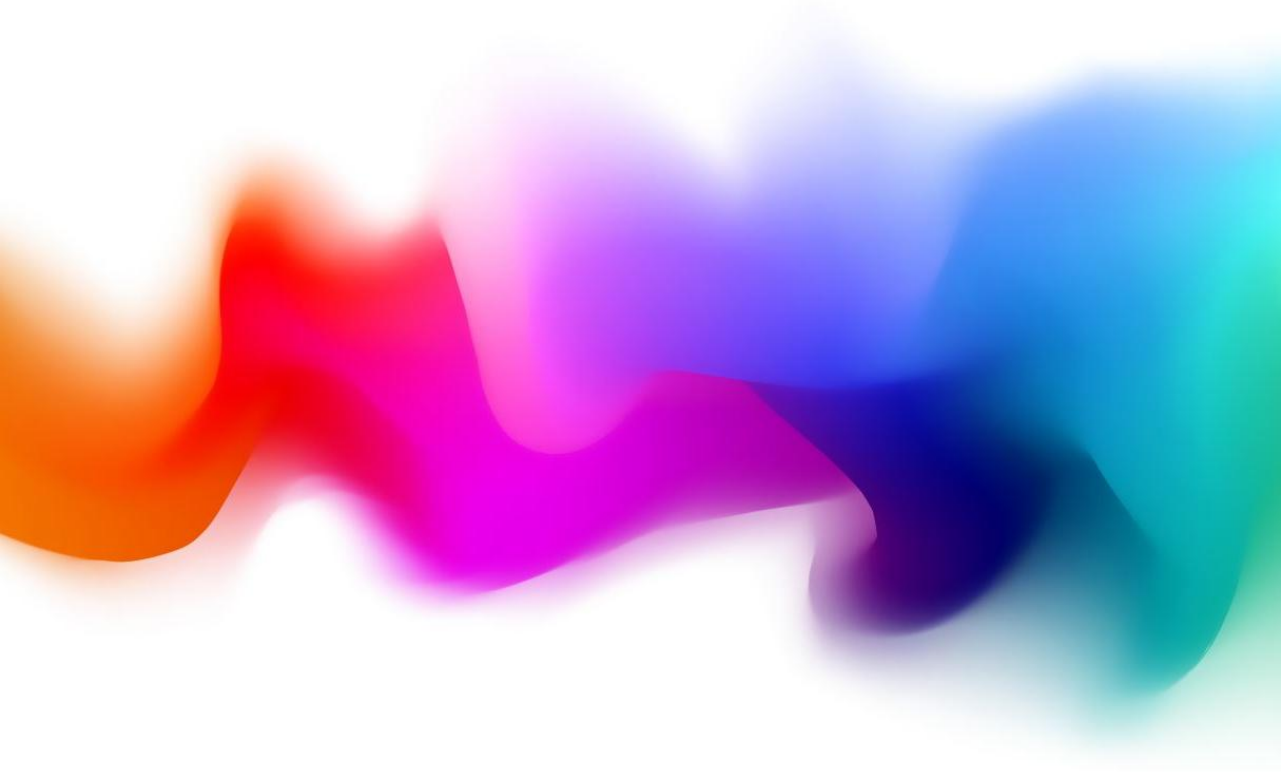

SQL Commands

V P JAYACHITRA

SQL Commands are divided into 5 categories:

Category	Full Form	Purpose	Examples
DDL	Data Definition Language	Define/modify database structure	CREATE, ALTER, DROP, TRUNCATE, RENAME
DML	Data Manipulation Language	Manipulate data in tables	INSERT, UPDATE, DELETE
DQL	Data Query Language	Retrieve data from database	SELECT
DCL	Data Control Language	Control access permissions	GRANT, REVOKE
TCL	Transaction Control Language	Manage transactions	COMMIT, ROLLBACK, SAVEPOINT

What is DDL?

Data Definition Language (DDL) statements are used to define the database structure or schema. DDL is used to create, modify, and remove database objects such as tables, indexes, and users.

Key Characteristics:

Changes database structure, not data

Auto-commits - cannot be rolled back

Updates system data dictionary

Creates metadata about database objects

DDL Commands Overview

1.CREATE - Build new database objects

Create tables, indexes, views, sequences

2. ALTER - Modify existing structure

Add/modify/drop columns

Add/drop constraints

3. DROP - Remove database objects completely

Deletes structure AND data

Cannot be undone

DDL Commands Overview

4. TRUNCATE - Remove all data, keep structure

Faster than DELETE

Cannot be rolled back

5. RENAME - Change object names

Tables, columns, constraints

CREATE TABLE

The CREATE TABLE command is used to specify a new base relation by giving it a name, and specifying each of its attributes and their data types, and any initial constraints on the relation

```
CREATE TABLE [schema.]table_name (
   column_name datatype [DEFAULT expr] [column_constraint],
   column_name datatype [DEFAULT expr] [column_constraint],
   ...
   [table_constraint]
);
```

What does CREATE TABLE do?

Defines table name
Specifies columns and their data types
Applies integrity constraints
Allocates storage space
Registers in data dictionary

CREATE TABLE-Example

```
CREATE TABLE Employee (
   Empno   NUMBER(4),
   Ename   VARCHAR2(50),
   Salary  NUMBER(8,2)
);
```

Data Types - Numeric Types

Numeric data types are used to store numeric values. The NUMBER datatype can store fixed-point and floating-point numbers

NUMBER(p,*s*) where p is precision (total digits) and s is scale (digits after decimal).

Syntax

```
NUMBER(4)      -- Integer, 0 to 9999
NUMBER(6,2)    -- Decimal, -9999.99 to 9999.99
NUMBER(10,2)   -- Decimal, -99999999.99 to 99999999.99
NUMBER         -- Floating point, any precision
```

Maximum value: 9999.99
9 9 9 9 . 9 9
Total 6 digits
4 digits before decimal (6-2=4)
digits after decimal

Understanding NUMBER Data Type

1: What is the maximum value that can be stored in NUMBER(5,1)?

A) 99999.9
B) 9999.9
C) 999.99
D) 99999

2: You need to store employee salaries up to 99,999.75. Which is correct?

A) NUMBER(5,2)
B) NUMBER(6,2)
C) NUMBER(7,2)
D) NUMBER(8,2)

Understanding NUMBER Data Type

1: What is the maximum value that can be stored in NUMBER(5,1)?

A) 99999.9
B) 9999.9
C) 999.99
D) 99999

2: You need to store employee salaries up to 99,999.75. Which is correct?

A) NUMBER(5,2)
B) NUMBER(6,2)
C) NUMBER(7,2)
D) NUMBER(8,2)

Data Types - Character Types

VARCHAR2(size) vs CHAR(size)

- **VARCHAR2:** ***Variable-length character string. More efficient than CHAR for variable- length data.***
- **CHAR:** ***Fixed-length character string. Blank-padded to specified length.***

Feature	CHAR(10)	VARCHAR2(10)
Storage for 'Raj'	10 bytes ('Raj ')	3 bytes ('Raj')
Storage for 'Vijay'	10 bytes ('Vijay ')	5 bytes ('Vijay')
Best for	Fixed-length codes	Variable-length text
Examples	Gender ('M'/'F'), Country code ('IN')	Names, addresses, emails

```
CHAR(1)        -- Gender: 'M' or 'F' (always 1 char)
CHAR(9)        -- SSN: Always 9 digits
VARCHAR2(50)   -- Name: 'Raj' or 'Vijayalakshmi'
VARCHAR2(100)  -- Address: Variable length
```

Why VARCHAR2? Why Not CHAR or VARCHAR?

CHAR(50): 50,000 bytes = 48.8 KB (lots of wasted space!)

VARCHAR2(50): ~10,000 bytes = 9.7 KB (only actual data)

MCQ - Understanding Character Types

1: How much storage does CHAR(20) use for storing 'HELLO'?

A) 5 bytes
B) 10 bytes
C) 20 bytes
D) Variable

2: Which is the BEST choice for storing employee names (length varies 3-50 chars)?

A) CHAR(50)
B) VARCHAR2(50)
C) VARCHAR(50)
D) CHAR(100)

MCQ - Understanding Character Types

1: How much storage does CHAR(20) use for storing 'HELLO'?

A) 5 bytes
B) 10 bytes
C) 20 bytes. CHAR is FIXED lengthtotal: 20 bytes regardless of actual text length
D) Variable

2: Which is the BEST choice for storing employee names (length varies 3-50 chars)?

A) CHAR(50)
B) VARCHAR2(50) VARCHAR2 uses only needed space (efficient)

C) VARCHAR(50)
D) CHAR(100)

Data Types - Date and Time

The DATE datatype is used to store date and time information, including century, year, month, day, hour, minute, and second.

What DATE stores: Storage: 7 bytes (fixed)

Year (4 digits)

Month (01-12)

Day (01-31)

Hour (00-23)

Minute (00-59)

Second (00-59)

```
Birthdate DATE
Hire_date DATE
Order_date DATE DEFAULT SYSDATE  -- Current date/time
```

```
Hire_date DATE
INSERT INTO Employee VALUES (101, 'Raj', DATE '2024-01-15');
```

MCQ - Understanding DATE

1.What does the DATE data type store?

A) Only date (YYYY-MM-DD)

B) Only time (HH:MI:SS)

C) Both date and time (to the second)

D) Date, time, and milliseconds

2. An employee's birthdate is 1995-05-10. Which is correct?

A) Birthdate VARCHAR2(10)

B) Birthdate DATE

C) Birthdate NUMBER

D) Birthdate CHAR(10)`

MCQ - Understanding DATE

What does the DATE data type store?

A) Only date (YYYY-MM-DD)

B) Only time (HH:MI:SS)

C) Both date and time (to the second) DATE stores: Year, Month, Day, Hour, Minute, Second. Example: 2024-01-15 14:30:45

D) Date, time, and milliseconds

An employee's birthdate is 1995-05-10. Which is correct?

A) `Birthdate VARCHAR2(10)`

B) `Birthdate DATE` Validates input: Rejects invalid dates. Supports date functions

C) `Birthdate NUMBER`

D) `Birthdate CHAR(10)`

CONSTRAINTS

Constraints are conditions that must hold on all valid relation instances. They specify rules that data must follow to maintain integrity and consistency in the database.

Purpose of Constraints:

1. **Data Integrity - Ensure data accuracy**
2. **Business Rules - Enforce company policies**
3. **Relationships - Maintain connections between tables**
4. **Validation- Prevent invalid data entry**

Types of Constraints:

Column-Level:	**Table-Level:**
PRIMARY KEY	PRIMARY KEY (composite)
UNIQUE	FOREIGN KEY
NOT NULL	UNIQUE (composite)
CHECK	CHECK (multi-column)
DEFAULT	

PRIMARY KEY Constraint

A primary key of a relation is an attribute or set of attributes that uniquely identifies each tuple. No two tuples can have the same value for primary key attributes, and primary key attributes cannot have NULL values.

- Column-level
- column_name datatype PRIMARY KEY
- Table-level
- CONSTRAINT constraint_name PRIMARY KEY (column_name)
- Composite (table-level only)
- CONSTRAINT constraint_name PRIMARY KEY (column1, column2)

PRIMARY KEY Constraint

A primary key of a relation is an attribute or set of attributes that uniquely identifies each tuple. No two tuples can have the same value for primary key attributes, and primary key attributes cannot have NULL values.

```
CREATE TABLE Employee (
   Empno   NUMBER(4) PRIMARY KEY,
   Ename   VARCHAR2(50),
   Salary  NUMBER(8,2)
);
```

Feature	PRIMARY KEY	UNIQUE
NULL allowed?	NO (always NOT NULL)	YES (can be NULL)
How many per table?	ONE only	Multiple allowed
Creates index?	YES (automatic)	YES (automatic)
Purpose	Main identifier	Alternative identifier

Why PRIMARY KEY?Why not Unique?

Why you NEED PRIMARY KEY:

MCQ - Understanding PRIMARY KEY

1: What does PRIMARY KEY automatically enforce?

A) UNIQUE only
B) NOT NULL only
C) UNIQUE + NOT NULL + Creates Index
D) UNIQUE + CHECK

2.How many PRIMARY KEYs can a table have?

A) Unlimited
B) One
C) Two
D) Depends on columns

MCQ - Understanding PRIMARY KEY

1: What does PRIMARY KEY automatically enforce?

A) UNIQUE only
B) NOT NULL only
C) UNIQUE + NOT NULL + Creates Index no duplicates, must have value, fast searching
D) UNIQUE + CHECK

How many PRIMARY KEYs can a table have?

A) Unlimited
B) One Fundamental rule: ONE primary key per table, Can be single column OR composite (multiple columns combined) PRIMARY KEY (Empno, Deptno)

C) Two
D) Depends on columns

FOREIGN KEY Constraint

A foreign key is a set of attributes in one relation that references the primary key of another relation. It is used to establish and enforce a link between the data in two tables.

```
Column-level
column_name datatype REFERENCES
parent_table(parent_column)

Table-level (recommended)
CONSTRAINT constraint_name
    FOREIGN KEY (column_name)
    REFERENCES parent_table(parent_column)
    [ON DELETE CASCADE | SET NULL | RESTRICT]
```

Purpose:
Maintain REFERENTIAL INTEGRITY
Foreign key value MUST exist in parent table
OR be NULL (if allowed)
Prevents "orphan" records

Rule: Child's FK value must exist in Parent's PK

Parent table (referenced)

```
CREATE TABLE Department (
   Deptno   NUMBER(2) PRIMARY KEY,
   Deptname VARCHAR2(30) NOT NULL
);
```

Child table (references parent)

```
CREATE TABLE Employee (
   Empno   NUMBER(4) PRIMARY KEY,
   Ename   VARCHAR2(50) NOT NULL,
   Deptno  NUMBER(2),  -- Foreign Key
   CONSTRAINT fk_emp_dept
      FOREIGN KEY (Deptno) REFERENCES Department(Deptno)
);
```

ON DELETE Actions - CASCADE vs SET NULL vs RESTRICT

Three Options When Parent is Deleted:

1. ON DELETE CASCADE - Delete children too

```
FOREIGN KEY (Deptno) REFERENCES Department(Deptno) ON DELETE CASCADE

DELETE Department WHERE Deptno=10;
→ Department 10 deleted
→ ALL employees in Dept 10 also DELETED!
```

ON DELETE Actions - CASCADE vs SET NULL vs RESTRICT

Three Options When Parent is Deleted:

2. ON DELETE SET NULL - Keep children, unassign them

```
FOREIGN KEY (Deptno) REFERENCES Department(Deptno) ON DELETE SET NULL

DELETE Department WHERE Deptno=10;
→ Department 10 deleted
→ Employees' Deptno set to NULL (unassigned)
```

ON DELETE Actions - CASCADE vs SET NULL vs RESTRICT

Three Options When Parent is Deleted:

3. ON DELETE RESTRICT (Default) - Block deletion

```
FOREIGN KEY (Deptno) REFERENCES Department(Deptno)
-> No ON DELETE = RESTRICT

DELETE Department WHERE Deptno=10;
→ ERROR! Cannot delete
→ Employees exist in this department
```

Why CASCADE? Why SET NULL? Why RESTRICT?

When to use CASCADE:

Use when child CANNOT exist without parent

Order → OrderItems: Delete order → delete its items
Post → Comments: Delete post → delete comments
Project → Tasks: Delete project → delete tasks

Why CASCADE? Why SET NULL? Why RESTRICT?

When to use SET NULL:

- *Use when child CAN exist independently (temporarily unassigned)* Department → Employee: Delete dept → unassign employees
  Category → Product: Delete category → uncategorize products
  Manager → Employee: Delete manager → orphan subordinates

Why CASCADE? Why SET NULL? Why RESTRICT?

When to use RESTRICT (default):

Use when deletion should be prevented (safest option)

- Customer → Orders: Can't delete customer with orders
- Student → Enrollments: Can't delete enrolled student
- Product → OrderItems: Can't delete ordered product

MCQ - Understanding FOREIGN KEY

1: What does a FOREIGN KEY constraint ensure?

A) No NULL values
B) No duplicate values
C) Value must exist in referenced table
D) Automatic index creation

2. With ON DELETE CASCADE, what happens when parent is deleted?

A) Only parent deleted
B) Parent and all its children deleted
C) Children's FK set to NULL
D) Deletion blocked

MCQ - Understanding FOREIGN KEY

1: What does a FOREIGN KEY constraint ensure?

A) No NULL values
B) No duplicate values
C) Value must exist in referenced table FK value must match a PK value in parent table
Prevents orphan records Can be NULL (unless NOT NULL specified separately)
D) Automatic index creation

2. With ON DELETE CASCADE, what happens when parent is deleted?

- A) Only parent deleted
  B) Parent and all its children deleted Like cutting tree branch - all leaves fall too
  C) Children's FK set to NULL
  D) Deletion blocked

UNIQUE Constraint

The UNIQUE constraint ensures that all values in a column or group of columns are distinct.

Unlike PRIMARY KEY, UNIQUE allows NULL values.

Column-level
column_name datatype UNIQUE

Table-level
CONSTRAINT constraint_name UNIQUE (column_name)

Composite UNIQUE
CONSTRAINT constraint_name UNIQUE (column1, column2)

Key Differences from PRIMARY KEY:

```
CREATE TABLE Employee (
Empno NUMBER(4) PRIMARY KEY,
Ename VARCHAR2(50) NOT NULL,
Email VARCHAR2(100) UNIQUE,
Phone VARCHAR2(15) UNIQUE,
Salary NUMBER(8,2)
);
```

Feature	PRIMARY KEY	UNIQUE
NULL allowed?	NO	YES
Per table	ONE only	MULTIPLE
Purpose	Main identifier	Alternative identifier

MCQ - Understanding UNIQUE

1: What's the main difference between PRIMARY KEY and UNIQUE?

A) PRIMARY KEY creates index, UNIQUE doesn't
B) PRIMARY KEY is faster
C) PRIMARY KEY cannot be NULL, UNIQUE can be NULL
D) A table can have multiple PRIMARY KEYs

2: How many UNIQUE constraints can a table have?

A) One only
B) Two
C) Same as PRIMARY KEY
D) Multiple

MCQ - Understanding UNIQUE

1: What's the main difference between PRIMARY KEY and UNIQUE?

A) PRIMARY KEY creates index, UNIQUE doesn't
B) PRIMARY KEY is faster
C) PRIMARY KEY cannot be NULL, UNIQUE can be NULL PRIMARY KEY = UNIQUE + NOT NULL UNIQUE alone allows NULL values Both create indexes automatically
D) A table can have multiple PRIMARY KEYs

2: How many UNIQUE constraints can a table have?

A) One only
B) Two
C) Same as PRIMARY KEY
D) Multiple PRIMARY KEY: Exactly ONE per table UNIQUE: As many as neede

NOT NULL Constraint

The NOT NULL constraint specifies that the attribute cannot have a NULL (unknown) value. NOT NULL is implicitly specified for attributes that are part of a primary key.

Only column-level (not table-level)
column_name datatype NOT NULL

What NOT NULL does:
Ensures column always has a value
Prevents NULL (unknown/missing) values
Must be specified at column level
PRIMARY KEY automatically includes NOT NULL

NOT NULL - Example

```
CREATE TABLE Employee (

Empno NUMBER(4) PRIMARY KEY, -- Automatically NOT NULL

Ename VARCHAR2(50) NOT NULL, -- Name is required Email

VARCHAR2(100), -- Email is optional

Salary NUMBER(8,2) NOT NULL, -- Salary is required

Commission NUMBER(8,2) -- Commission is optional );
```

MCQ - Understanding NOT NULL

1: Can NOT NULL constraint be specified at table level?

A) Yes, both column and table level
B) No, only at column level
C) Yes, but only in Oracle 12c+
D) Depends on datatype

2: What happens when you insert an empty string '' into a NOT NULL column in Oracle?

A) Accepted as empty string
B) Error - NULL not allowed
C) Converted to space ' '
D) Accepted as empty

MCQ - Understanding NOT NULL

1: Can NOT NULL constraint be specified at table level?

A) Yes, both column and table level
B) No, only at column level
C) Yes, but only in Oracle 12c+ NOT NULL is always column-level
D) Depends on datatype

2: What happens when you insert an empty string '' into a NOT NULL column in Oracle?

A) Accepted as empty string
B) Error - NULL not allowed Oracle treats '' (empty string) as NULL NOT NULL constraint rejects NULL
C) Converted to space ' '
D) Accepted as empty

CHECK Constraint

The CHECK clause specifies a condition that each tuple must satisfy. The condition is a boolean expression that is evaluated for each tuple, and the tuple is accepted only if the condition evaluates to TRUE

Column-level
column_name datatype CHECK (condition)

Table-level
CONSTRAINT constraint_name CHECK (condition)

What CHECK does:
Validates data based on custom rules
Enforces business logic
Can reference same row's columns
Cannot contain subqueries or reference other tables

Example :Check Constraint

```
CREATE TABLE Employee (
    Empno  NUMBER(4) PRIMARY KEY,
    Ename  VARCHAR2(50) NOT NULL,
    Age    NUMBER(3) CHECK (Age >= 18 AND Age <= 65),
    Salary NUMBER(8,2) CHECK (Salary > 0),
    Gender CHAR(1) CHECK (Gender IN ('M', 'F', 'O')),
    Commission NUMBER(8,2),

    -- Table-level CHECK (multi-column)
    CHECK (Commission IS NULL OR Commission <= Salary)
);
```

MCQ - Understanding CHECK

1: Which CHECK constraint is VALID?

A) CHECK (Salary > (SELECT AVG(Salary) FROM Employee))
B) CHECK (Salary > 0 AND Salary < 100000)
C) CHECK (Deptno IN (SELECT Deptno FROM Department))
D) CHECK (SYSDATE < Hire_date)

2. Given CHECK (Commission <= Salary), what happens with Salary=50000, Commission=NULL?

A) Rejected - NULL is not <= 50000
B) Accepted - NULL satisfies CHECK
C) Error - Commission cannot be NULL
D) Depends on NOT NULL constraint

MCQ - Understanding CHECK

1: Which CHECK constraint is VALID?

A) CHECK (Salary > (SELECT AVG(Salary) FROM Employee))
B) CHECK (Salary > 0 AND Salary < 100000)
C) CHECK (Deptno IN (SELECT Deptno FROM Department))
D) CHECK (SYSDATE < Hire_date)

- CHECK can use constants and operators
- Cannot use subqueries (A, C)
- Cannot use non-deterministic functions like SYSDATE (D)
- Can compare column to constants and use AND/OR

MCQ - Understanding CHECK

2. Given CHECK (Commission <= Salary), what happens with Salary=50000, Commission=NULL?

A) Rejected - NULL is not <= 50000
B) Accepted - NULL satisfies CHECK
C) Error - Commission cannot be NULL
D) Depends on NOT NULL constraint

Explanation:
•NULL comparisons result in UNKNOWN
•UNKNOWN satisfies CHECK constraint
•Only FALSE rejects the row
•To prevent NULL, must use NOT NULL separately

DEFAULT Constraint - Definition

The DEFAULT clause specifies a default value for a column. When an INSERT statement omits the column, the default value is automatically assigned.

```
column_name datatype DEFAULT default_value
```

Valid DEFAULT values:
Literal constants: DEFAULT 0, DEFAULT 'ACTIVE'
System functions: DEFAULT SYSDATE, DEFAULT USER
Cannot use: Subqueries, column references, aggregate functions

DEFAULT Constraint - Definition

```
CREATE TABLE Employee (
   Empno  NUMBER(4) PRIMARY KEY,
   Ename  VARCHAR2(50) NOT NULL,
   Status VARCHAR2(10) DEFAULT 'ACTIVE',
   Hire_date DATE DEFAULT SYSDATE,
   Login_count NUMBER DEFAULT 0,
   Department VARCHAR2(30) DEFAULT 'Unassigned',
   Is_active CHAR(1) DEFAULT 'Y'
);
```

Combining DEFAULT with NOT NULL:

```
CREATE TABLE Employee (
Empno NUMBER(4) PRIMARY KEY,
Status VARCHAR2(10) NOT NULL DEFAULT 'ACTIVE'
);
```

MCQ - Understanding DEFAULT

1: Which DEFAULT value is VALID in Oracle?

A) DEFAULT (SELECT MAX(Salary) FROM Employee)
B) DEFAULT SYSDATE
C) DEFAULT Manager_salary
D) DEFAULT AVG(Salary)

2: What's the difference between omitting a column vs. setting it to NULL?

A) No difference
B) Omitting uses DEFAULT; NULL ignores DEFAULT
C) Both use DEFAULT
D) Both store NULL

MCQ - Understanding DEFAULT

1: Which DEFAULT value is VALID in Oracle?

A) DEFAULT (SELECT MAX(Salary) FROM Employee)

B) DEFAULT SYSDATE Cannot use subqueries (A) Cannot reference other columns (C) Cannot use aggregate functions (D)

C) DEFAULT Manager_salary

D) DEFAULT AVG(Salary)

2: What's the difference between omitting a column vs. setting it to NULL?

A) No difference

B) Omitting uses DEFAULT; NULL ignores DEFAULT

C) Both use DEFAULT

D) Both store NULL

Constraint	Purpose	NULL?	Per Table	Level
PRIMARY KEY	Main unique identifier	NO	ONE	Column/Table
FOREIGN KEY	Link to another table	YES*	Many	Table
UNIQUE	Alternative identifier	YES	Many	Column/Table
NOT NULL	Must have value	NO	Many	Column only
CHECK	Custom validation	YES*	Many	Column/Table
DEFAULT	Auto-fill value	YES	Many	Column only

*Unless NOT NULL specified separately

ALTER TABLE

The ALTER TABLE command is used to modify the structure of an existing table. Operations include adding/dropping columns, modifying column definitions, and adding/dropping constraints.

```
ALTER TABLE table_name
    ADD (column_definition | constraint)
    | MODIFY column_definition
    | DROP COLUMN column_name
    | DROP CONSTRAINT constraint_name
    | RENAME COLUMN old_name TO new_name;
```

Common Operations:

1. **ADD** - Add new columns or constraints
2. **MODIFY** - Change column definition
3. **DROP** - Remove columns or constraints
4. **RENAME** - Change column names

ALTER TABLE-Add

Add single column

```
ALTER TABLE Employee ADD Email VARCHAR2(100);
```

Add multiple columns

```
ALTER TABLE Employee ADD (
   Phone VARCHAR2(15),
   City VARCHAR2(30) DEFAULT 'Mumbai'
);
```

Add column with constraint

```
ALTER TABLE Employee ADD Mobile VARCHAR2(15) UNIQUE;
```

WITH DEFAULT

```
ALTER TABLE Employee ADD Status VARCHAR2(10) DEFAULT 'ACTIVE';
-- Existing rows get DEFAULT value!
```

ALTER TABLE-Modify column

Increase size (always safe)

```
ALTER TABLE Employee MODIFY Ename VARCHAR2(100);
```

Decrease size (check data first!)

```
ALTER TABLE Employee MODIFY Ename VARCHAR2(30);
```

Add NOT NULL (ensure no NULLs first!)

```
ALTER TABLE Employee MODIFY Email VARCHAR2(100) NOT NULL;
```

Change datatype (usually requires empty column)

```
ALTER TABLE Employee MODIFY Empno VARCHAR2(10);
```

ALTER TABLE-Modify column

ALWAYS SAFE:

- Increase size: VARCHAR2(50) → VARCHAR2(100)

- Increase precision: NUMBER(6,2) → NUMBER(10,2)

- Remove NOT NULL: NOT NULL → allow NULL

CHECK DATA FIRST:

- **Decrease size: VARCHAR2(100) → VARCHAR2(50)**
- SELECT MAX(LENGTH(Ename)) FROM Employee;
- *NOTE: If result <= 30, safe to decrease to VARCHAR2(30)*

- Decrease precision: NUMBER(10,2) → NUMBER(6,2)

- Add NOT NULL: allow NULL → NOT NULL

- Change datatype: VARCHAR2 → NUMBER

ALTER TABLE - DROP Operations

DROP Column:

Drop single column

ALTER TABLE Employee DROP COLUMN Email;

Drop multiple columns (not supported, do separately)

ALTER TABLE Employee DROP COLUMN Phone;

ALTER TABLE Employee DROP COLUMN City;

DROP Constraint:

Find constraint name first:

```
SELECT constraint_name
FROM user_constraints
WHERE table_name = 'EMPLOYEE';
```

Drop constraint by name:

```
ALTER TABLE Employee DROP CONSTRAINT chk_salary;
```

Drop PRIMARY KEY:

```
ALTER TABLE Employee DROP PRIMARY KEY;
```

Drop PRIMARY KEY with cascading FKs:

```
ALTER TABLE Employee DROP PRIMARY KEY CASCADE;
```

Cannot drop:

- Last remaining column
- Column used in constraint (drop constraint first)
- All constraints at once (do one by one)

DROP is permanent!

- Cannot be rolled back (DDL auto-commits)
- Make backup first!

ALTER TABLE - ADD Constraint Examples

Add PRIMARY KEY:

```
ALTER TABLE Employee
ADD CONSTRAINT pk_emp PRIMARY KEY (Empno);
```

Add FOREIGN KEY:

```
ALTER TABLE Employee
ADD CONSTRAINT fk_emp_dept
FOREIGN KEY (Deptno) REFERENCES Department(Deptno);
```

Add UNIQUE:

```
ALTER TABLE Employee
ADD CONSTRAINT uq_email UNIQUE (Email);
```

- *Before adding constraint, verify data:*

```
SELECT COUNT(*) FROM Employee WHERE Salary <= 10000;
```

If count > 0, fix data first or constraint will fail!

Add CHECK:

```
ALTER TABLE Employee
ADD CONSTRAINT chk_salary CHECK (Salary > 10000);
```

Add NOT NULL (use MODIFY):

```
ALTER TABLE Employee
MODIFY Ename VARCHAR2(50) NOT NULL;
```

MCQ

1: You have a table with Salary values: 5000, 15000, 20000. What happens when you execute: ALTER TABLE Employee ADD CHECK (Salary > 10000);

A) Constraint added, all data passes validation
B) Constraint added, only validates new data
C) Error: Constraint violated, not added
D) Constraint added, invalid data deleted

MCQ

1: You have a table with Salary values: 5000, 15000, 20000. What happens when you execute: ALTER TABLE Employee ADD CHECK (Salary > 10000);

A) Constraint added, all data passes validation
B) Constraint added, only validates new data
C) Error: Constraint violated, not added
D) Constraint added, invalid data deleted

DROP TABLE - Complete Removal:

Delete table structure AND data

DROP TABLE Employee;

Drop with cascading constraints:

DROP TABLE Employee CASCADE CONSTRAINTS;

TRUNCATE TABLE - Remove All Data:

Delete all rows, keep structure

TRUNCATE TABLE Employee;

- **Faster than DELETE FROM Employee**
- **Cannot be rolled back!**
- **Resets auto-increment counters**

RENAME TABLE:

Rename entire table
RENAME Employee TO Staff;

Rename column (within ALTER TABLE)
ALTER TABLE Employee RENAME COLUMN Ename TO Employee_name;

Comparison:

Command	Structure	Data	Rollback?	Speed
DROP	Deleted	Deleted	NO	Fast
TRUNCATE	Kept	Deleted	NO	Fastest
DELETE	Kept	Deleted	YES	Slow

Employee Database Exercise

Every employee must belong to a department

Every department has a manager (who is an employee)

Employees can work on multiple projects

Projects belong to departments

Track hours employees spend on projects

EMPLOYEE

Table Name: EMP + last 4 digits of your register number

Example: If register number is 21CS5001, create table EMP5001

EMPLOYEEE

Table-Level Constraints:

- Commission should not be greater than Salary
- Date of joining should not be a future date

Column Name	Attribute Specification	Constraint
Empno	4-digit number	PRIMARY KEY, Must be unique
Empname	Up to 50 characters	NOT NULL
Deptno	2-digit number	Must be between 10 and 90
Jobtitle	Up to 50 characters	Must be one of: 'MANAGER', 'ANALYST', 'CLERK', 'SALESMAN'
DoJ	Date value	Date of joining should not be a future date
Salary	6 digits before decimal, 2 after	Must be greater than 0
Commission	6 digits before decimal, 2 after	Should not be greater than salary, Can be NULL

DEPARTMENT

Table-Level Constraints:

- Deptno must be same range as EMPLOYEE.Deptno (10-90)
- Manager_empno is FOREIGN KEY to EMPLOYEE(Empno) with ON DELETE SET NULL

column Name	Attribute Specification	Constraint
Deptno	2-digit number	PRIMARY KEY
Deptname	Up to 30 characters	NOT NULL, UNIQUE
Location	Up to 30 characters	Optional (can be NULL)
Manager_empno	4-digit number	References EMPLOYEE(Empno), Can be NULL initially

PROJECT

Table-Level Constraints:

- End_date must be NULL OR greater than Start_date
- Deptno references DEPARTMENT with ON DELETE CASCADE

Column Name	Attribute Specification	Constraint
Projectno	4-digit number	PRIMARY KEY
Projectname	Up to 50 characters	NOT NULL, UNIQUE
Deptno	2-digit number	References DEPARTMENT(Deptno), NOT NULL
Budget	10 digits before decimal, 2 after	Must be greater than 0
Start_date	Date value	NOT NULL, Must not be future date
End_date	Date value	Can be NULL (ongoing projects), Must be after Start_date if provided

Step 1: Create table with column-level constraints

```
CREATE TABLE EMP5001 (

Empno NUMBER(4) CONSTRAINT pk_emp5001 PRIMARY KEY,

Empname VARCHAR2(50) CONSTRAINT nn_empname NOT NULL,

Deptno NUMBER(2) CONSTRAINT chk_deptno CHECK (Deptno BETWEEN 10 AND 90),

Jobtitle VARCHAR2(50),

DoJ DATE,

Salary NUMBER(8,2) CONSTRAINT chk_salary_positive  CHECK (Salary > 0),

Commission NUMBER(8,2)
);
```

Step 2: Add job title constraint

```
ALTER TABLE EMP5001
ADD CONSTRAINT chk_jobtitle
CHECK (Jobtitle IN ('MANAGER', 'ANALYST', 'CLERK', 'SALESMAN'));
```

Step 3: Add commission <= salary constraint

```
ALTER TABLE EMP5001
ADD CONSTRAINT chk_commission
CHECK (Commission IS NULL OR Commission <= Salary);
```

Step 4: Add DoJ future date check

```
ALTER TABLE EMP5001
ADD CONSTRAINT chk_doj
CHECK (DoJ <= SYSDATE);
```

```
create table department5001 (
deptno number(2) constraint pk_dept5001 primary key
constraint chk_dept_range check (deptno between 10 and 90),
deptname varchar2(30) constraint nn_deptname not null
constraint uq_deptname unique,
location varchar2(30),
manager_empno number(4) );
```

- **add foreign key after employee table exists**

```
alter table department5001
add constraint fk_dept_manager
 foreign key (manager_empno) references emp5001(empno) on
delete set null;
```

```
CREATE TABLE PROJECT (
Projectno NUMBER(4) CONSTRAINT pk_project5001 PRIMARY KEY,

Projectname VARCHAR2(50) CONSTRAINT nn_projname NOT NULL
CONSTRAINT uq_projname UNIQUE,

Deptno NUMBER(2) CONSTRAINT nn_proj_dept NOT NULL,

Budget NUMBER(12,2) CONSTRAINT chk_budget CHECK (Budget > 0),

Start_date DATE CONSTRAINT nn_start_date NOT NULL
CONSTRAINT chk_start_date CHECK (Start_date <= SYSDATE),
End_date DATE,
CONSTRAINT chk_dates CHECK (End_date IS NULL OR End_date > Start_date),

CONSTRAINT fk_proj_dept
      FOREIGN KEY (Deptno) REFERENCES DEPARTMENT5001(Deptno)  ON DELETE CASCADE
);
```

DEPARTMENT Table

Specifications:

Column	Datatype	Constraints
Deptno	NUMBER(2)	PRIMARY KEY, 10-90
Deptname	VARCHAR2(30)	NOT NULL, UNIQUE
Location	VARCHAR2(30)	Optional
Manager_empno	NUMBER(4)	FK to EMPLOYEE, SET NULL on delete

PROJECT Table

Specifications:

Column	Datatype	Constraints
Projectno	NUMBER(4)	PRIMARY KEY
Projectname	VARCHAR2(50)	NOT NULL, UNIQUE
Deptno	NUMBER(2)	FK to DEPARTMENT, CASCADE on delete
Budget	NUMBER(12,2)	CHECK > 0
Start_date	DATE	NOT NULL
End_date	DATE	NULL or > Start_date